

Received June 22, 2017, accepted July 19, 2017, date of publication August 17, 2017, date of current version October 12, 2017.

Digital Object Identifier 10.1109/ACCESS.2017.2740418

A Quality and Distance Guided Metaheuristic Algorithm for Vertex Separation Problem

JUNWEN DING, TAOQING ZHOU, ZHIPENG LÜ, AND YUJIN YUAN

School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

Corresponding author: Zhipeng Lü (zhipeng.lv@hust.edu.cn)

This work was supported in part by the National Natural Science Foundation of China under Grant 61370183 and in part by the Programme for New Century Excellent Talents in University under Grant NCET-12-0219.

ABSTRACT This paper presents a quality and distance guided metaheuristic algorithm (QD-ILS) for solving the vertex separation problem. QD-ILS integrates a basic local search procedure with QD-LS strategy, which uses an augmented evaluation function that considers both solution quality and distance between the current solution and the best found solution to guide the search to explore promising regions of the search space. Assessed on two sets of 162 common benchmark instances, QD-ILS achieves highly competitive results in terms of both solution quality and computational efficiency compared with the state-of-the-art algorithms in the literature. Specifically, it improves the previous best known results for 33 out of 162 benchmark instances and matches the best known results on all except four of the remaining instances compared with the state-of-the-art algorithms in the literature. The impact of the distance and quality-based diversification strategy is also investigated.

INDEX TERMS Vertex separation problem, hybrid metaheuristic algorithm, quality and distance guided local search.

I. INTRODUCTION

Given an undirected and unweighted graph $G(V, E)$ where V (with $|V| = n$) and E (with $|E| = m$) are the sets of vertices and edges, respectively. A linear layout φ of the vertices of G is a mapping $\varphi : V \rightarrow \{1, \dots, n\}$ in which each vertex receives a unique and different integer between 1 and n . For vertex u , let $\varphi(u)$ denote its position or label in layout φ . Let $L(p, \varphi, G)$ be the set of vertices in V with a position in the layout φ lower than or equal to position p . Symmetrically, let $R(p, \varphi, G)$ be the set of vertices with a position in the layout φ larger than position p . Formally, $L(p, \varphi, G) = \{v | v \in V, \varphi(v) \leq p\}$ and $R(p, \varphi, G) = \{v | v \in V, \varphi(v) > p\}$.

Since layouts are usually represented in a straight line, $L(p, \varphi, G)$ can be simply regarded as the set of left vertices with respect to position p , and $R(p, \varphi, G)$ is the set of right vertices with respect to position p . The Cut-value at position p of layout φ , $Cut(p, \varphi, G)$, is defined as the number of vertices in $L(p, \varphi, G)$ with one or more adjacent vertices in $R(p, \varphi, G)$. Then, $Cut(p, \varphi, G) = |\{u | u \in L(p, \varphi, G), v \in R(p, \varphi, G), (u, v) \in E\}|$.

The vertex separation (VS) value of layout φ is the maximum of the Cut-value among all positions in layout φ : $VS(\varphi, G) = \max_p Cut(p, \varphi, G)$. The vertex separation problem (VSP) consists of finding a layout, say φ^* , minimizing the VS value in φ of graph G .

Besides its theoretical significance as a canonical NP-hard problem [1], the VSP is notable for its ability to formulate a wide range of important problems such as the very large scale integration (VLSI) design, the computer language compiler design [2], [3], and the graph drawing [4]–[6]. Moreover, the VSP is highly related to other well-known NP-hard graph problems, such as the path-width problem [7], the node search number problem [8], the interval thickness problem [9]–[12], etc. The similarities and differences among these problems can be found in [7], [8], and [13].

Several exact approaches for tackling the VSP have been presented in the literature. Ellis et al. [14] proposed a linear algorithm to calculate the optimal vertex separation of a tree and an $O(n \lg n)$ algorithm to find the corresponding optimal layout. Later, Skodinis [15] improved the algorithm with a linear time procedure to find the optimal layout. Peng et al. [16] proposed an alternative method to calculate the vertex separation of trees. Ellis and Markov [17] introduced an $O(n \lg n)$ algorithm to calculate the vertex separation of unicyclic graphs (i.e., trees with an extra edge). Bodlaender and Möhring [18] proposed a linear time algorithm to optimally solve the cographs and permutation graphs. Bollobás and Leader [19] proposed a polynomial-time algorithm to optimally solve the VSP for n -dimensional grids. Later, Bodlaender et al. [20] provided

a polynomial-time algorithm to calculate the path-width of permutation graphs. Bodlaender et al. [21] gave experimental and theoretical results of the treewidth of a graph by exponential time algorithms using exponential or polynomial space. Bodlaender et al. [22] provided a proof that several vertex ordering problems can be solved in $O(2^n)$ time and $O(2^n)$ space, or in $O(4^n)$ time and polynomial space. Crescenzi et al. [23] proposed a new algorithm for the classical problem of computing the diameter of undirected unweighted graphs. Kobayashi et al. [24] evaluated the potential of commitments in heuristic algorithms for the path-width of undirected graphs. Coudert et al. [25] designed a branch-and-bound algorithm that computes the exact path-width of graphs and a corresponding path decomposition. Li et al. [26] analyzed the average clustering coefficient of graphs with large average degree, which also exists for strongly regular graphs.

Besides, some approximation algorithms have also been proposed for the VSP. Specifically, Bodlaender et al. [27] proposed a polynomial time $O(\lg^2 n)$ -approximation algorithm for general graphs and an $O(\lg n)$ -approximation algorithm for planar graphs. Similar results for binomial random graphs were presented in [28].

On the other hand, heuristic and metaheuristic methods, which have shown to be very powerful for various NP-hard combinatorial optimization problems, have seldomly been considered for VSP. In particular, Duarte et al. [29] proposed a basic variable neighborhood search (VNS) algorithm for the VSP which introduced two constructive procedures and one local search technique based on interchange moves. Later, Sánchez-Oro et al. [30] proposed different heuristic methods and embedded them into the variable neighborhood search scheme to solve this problem, which, to the best of our knowledge, showed excellent performance and obtained the best results for this problem up to now.

This paper presents QD-ILS, a quality and distance guided iterated local search algorithm, to solve the vertex separation problem. QD-ILS integrates a basic local search procedure with QD-LS strategy, which uses the augmented evaluation function that considers both solution quality and distance between the current solution and the best found solution to guide the search to explore promising regions of the search space. QD-ILS has a particular focus on the importance of the perturbation phase. After a local optimum is obtained by the local search phase, QD-LS is adopted to perturb the best found solution to generate a new starting solution for the next round of local search procedure.

QD-LS is a recently proposed diversification mechanism which has been successfully applied to solve the vertex separator problem [31]. The main idea of QD-LS is to apply local search with modified objective function to jump out of the poor local optima. Guided by an augmented evaluation function, QD-LS can introduce a proper dose of diversification by generating solutions that differ from the previously visited local optima with a suitable distance. Compared with conventional diversification mechanisms, QD-LS is a novel

diversification mechanism which can strive for a proper trade-off between intensification and diversification.

The main contributions of this paper are summarized as follows:

- To our knowledge, this work is the first to employ the quality and distance guided metaheuristic algorithm to solve the vertex separation problem.
- QD-ILS uses the quality and distance guided local search as a diversification mechanism.
- The outcomes show the efficacy of this algorithm in terms of both solution quality and computational efficiency. In particular, the proposed QD-ILS can improve the previous best known results for 33 out of 162 benchmark instances and match the best known results on all except 4 of the remaining instances compared with the state-of-the-art algorithms in the literature.

The rest of this paper is organized as follows. Section II presents the general principle of QD-LS and the main scheme and each component of QD-ILS. Section III reports the computational results of QD-ILS and compares with the state-of-the-art algorithms for VSP in the literature, where the key features of QD-ILS are also analyzed to identify its critical success factors. Section IV concludes the paper and suggests future research directions.

II. QD-LS GUIDED METAHEURISTIC ALGORITHM

In this section, we first introduce the principle of the proposed diversification mechanism: quality and distance guide local search, and then present the main scheme of the QD-ILS algorithm to tackle the VSP, followed by the detailed description of each component of QD-ILS.

A. QUALITY AND DISTANCE GUIDED LOCAL SEARCH

Quality and distance guided local search method is a local search based diversification mechanism, which is usually jointly used with other intensification-oriented algorithms to form an integrated optimization algorithm. QD-LS uses an augmented evaluation function F instead of the original objective function f to guide the search to explore promising regions of the search space [31]. The key component of the QD-LS concerns the augmented evaluation function F which takes the following form:

$$F(x) = f(x) + \beta \cdot d(x, x^*) \quad (1)$$

where $f(x)$ is the objective function value of solution x , $d(x, x^*)$ is the distance between the current solution x and the best found solution x^* , and β is the regularization parameter. For a minimization problem, β is negative (as in this paper $\beta = -4.5$), and the objective is to minimize F . Therefore, minimizing $f(x)$ and maximizing $d(x, x^*)$ are simultaneously reached, which can optimize the objective value $f(x)$ as far as possible while keeping the current solution apart from the previous best found solution. On the other hand, for a maximization problem, the objective is to maximize F , β should be positive and both $f(x)$ and $d(x, x^*)$ are maximized.

The rationale of QD-LS is to help the search to gradually move away from local optimum by changing the search landscape. The distance function $d(x, x^*)$ is the essential part of our QD-LS algorithm. The bias toward solution quality or distance between the current solution and the best found solution can be tuned by the parameter β . Considering both solution quality and distance between the current solution and the best found solution, QD-LS can obtain a solution not only with high quality but also different from the best found solution with a suitable distance. QD-LS is a diversification component of metaheuristic approaches, which can be embedded into the framework of other metaheuristics, such as tabu search, simulated annealing, variable neighborhood search, etc.

The proposals that most closely resemble the ‘quality and distance’ component of our approach come from the adaptive memory projection methods in [32] and the use of ‘target objectives’ in [33]. The projection methods in [32] introduces several types of distance measures for problems in bounded integer variables, which are embodied in ‘pseudo-cut’ inequalities to guide the search for diversification. Rather than these inequalities explicitly, our approach is equivalent to creating a Lagrangian objective function where β is the Lagrangian multiplier for the associated pseudo-cut. The ‘target objective’ approach of [33], by contrast, takes one or more such inequalities into the objective function by expressing them in the form of penalized goal programming variables rather than as a Lagrangian. To the best of our knowledge, these proposals have not been studied computationally.

B. MAIN SCHEME OF QD-ILS

QD-ILS is a general stochastic local search method which follows the basic scheme of the iterated local search (ILS). Its basic idea is to use a descent-based local search procedure to intensify the search within a given search space region, and to apply effective perturbations to move towards a new search region once a local optimum is obtained.

Algorithm 1 Pseudo-Code of QD-ILS for VSP

```

1: Input: The undirected graph  $G$ 
2: Output: The best partition  $\varphi^*$  found so far
3:  $\varphi \leftarrow \text{Init\_Solution}()$  /* Section II-C */
4:  $\varphi^* \leftarrow \varphi$ 
5: while stopping condition not met do
6:    $\varphi \leftarrow \text{Local\_Search}(\varphi)$  /* Section II-F */
7:   if  $VS(G, \varphi) < VS(G, \varphi^*)$  then
8:      $\varphi^* \leftarrow \varphi$ 
9:   end if
10:   $\varphi \leftarrow \text{QD\_Local\_Search}(\varphi^*)$  /* Section II-G */
11: end while

```

The general architecture of QD-ILS is described in Algorithm 1. It is mainly composed of three components: initial solution generator, a basic local search procedure, and a quality and distance guided local search procedure.

Starting with an initial solution, QD-ILS uses the basic local search procedure to optimize the current solution to reach a local optimum φ (line 6). The best found solution is recorded as φ^* (lines 7-9). Then, a quality and distance guided local search is employed on φ^* to help the search to escape from the local optima (line 10, i.e., to perturb the best found solution). These two procedures are repeated until the stopping condition is met. It can be seen that, in this iterated local search framework, the basic local search procedure aims to intensify the search to explore areas of search space with high quality solutions, while the quality and distance guided local search diversifies the search to unexplored areas when trapped into local optima. These two procedures are alternately launched in order to obtain a tradeoff between search intensification and search diversification.

C. INITIAL SOLUTION

In QD-ILS, we adopt the second constructive procedure in [29] to generate the initial solution. This procedure is based on the concept of level structures [34] where the set of vertices V is partitioned into different sets $L_1, \dots, L_l$ called levels. The first level, L_1 , contains only one vertex. The rest of levels L_l with $l = 2, 3, \dots, \lambda$ (where λ indicates the number of levels) contains all the vertices adjacent to some vertices in L_{l-1} that are not placed in any L_j with $1 \leq j < l$. The number of levels λ exclusively depends on the graph and the vertex placed in L_1 , and the vertices in alternative levels are not adjacent.

The initial solution is generated in the following way: First, starting with each vertex, we use the breadth first search strategy to traverse the graph, thus produce n different search trees. Second, we take the traversing order of the vertices of the tree with the largest level (i.e., the largest value of λ) as the initial solution.

D. NEIGHBORHOOD STRUCTURE

The neighborhood structure is the key component of any neighborhood-based search method. The solution of the VSP is typically represented as permutations of all the vertices. Two moves are frequently used in metaheuristic algorithms to generate neighborhood solutions for permutation problems: *swap* and *insert*.

Given a solution $\varphi = (v_1, \dots, v_i, \dots, v_j, \dots, v_n)$ for VSP where $v_i \in V, i = 1, \dots, n$. Let $\varphi(v_i)$ denote the position of vertex v_i in φ , i.e., $\varphi(v_i) = i$. Therefore, a *swap*(φ, i, j) move is defined as exchanging the vertices v_i and v_j , thus, producing a new solution $\varphi' = (v_1, \dots, v_j, \dots, v_i, \dots, v_n)$. Moreover, an *insert*(φ, i, j) (where $i < j$) move is defined as removing vertex v_i from its current position i , and inserting it to the position directly behind vertex v_j , which produces a neighboring solution $\varphi' = (v_1, \dots, v_{i-1}, v_{i+1}, \dots, v_j, v_i, \dots, v_n)$.

The size of both the *swap* and *insert* neighborhoods are $O(n^2)$. Considering the fact that it is time-consuming to evaluate all the possible moves, we adopt the first-improvement strategy rather than the best-improvement

strategy to move from one solution to another, implying that the move is executed once if it can improve the current solution.

E. CRITERION OF NEIGHBORHOOD SELECTION

As indicated in Section II-D, we adopt the first-improvement strategy in the *swap* and *insert* neighborhoods. The essential concept in the defined neighborhood is “improvement”. Here, we use the same criterion proposed in [29] (Also see Algorithm 2).

Algorithm 2 IsImprovementMove(φ, φ')

```

1: if  $VS(G, \varphi') < VS(G, \varphi)$  then
2:   return true
3: else if  $VS(G, \varphi') > VS(G, \varphi)$  then
4:   return false
5: else
6:   for  $i \leftarrow VS(G, \varphi)$  downto 1 do
7:     if  $|S_i(\varphi')| < |S_i(\varphi)|$  then
8:       return true
9:     else if  $|S_i(\varphi')| > |S_i(\varphi)|$  then
10:      return false
11:   end if
12: end for
13: return false
14: end if

```

Algorithm 2 shows the pseudo-code of the “improvement” criterion, where $S_i(\varphi)$ is the set of positions at which the Cut-value equals to i in φ , i.e., $S_i(\varphi) = \{p | \text{Cut}(p, \varphi, G) = i, p = 1, \dots, n-1\}$. This algorithm compares the objective function value and the ordering after the corresponding move. If the associated move reduces the objective function value, it returns *true*. On the other hand, if the move deteriorates the objective function value it returns *false*. If the move does not affect the objective function value, the algorithm traverses the set $S_i(\varphi)$ in descending order of i (i.e., i is from $VS(G, \varphi)$ to 1). It returns *true* or *false* if the number of positions at which the Cut-value equalling to i decreases or increases, respectively. Although this kind of move does not reduce the VS value of the graph, it reduces the number of positions with large Cut-value, which has the potential to further improve the solution.

F. BASIC LOCAL SEARCH PROCEDURE

The basic local search procedure, which is used to intensify the search, operates with the swap and insert neighborhoods defined in Section II-D. From a given solution, it iteratively improves the current solution by performing the profitable moves. Algorithm 3 gives the details of this procedure.

Algorithm 3 first traverses the n -sized swap neighborhood of the given solution φ (lines 5 to 15). Specifically, for each vertex v_i in position i of φ , we randomly select a vertex v_j from $V \setminus \{v_i\}$, then the swap move $\text{swap}(i, j)$ is performed on φ (denoted by $\oplus$) to produce a new solution φ' . If φ' is

Algorithm 3 Pseudo-Code of the Basic Local Search Procedure (LS)

```

1: Input: The graph  $G$ , the best found solution  $\varphi^*$ , and the solution  $\varphi$  to be optimized
2: Output: An optimized solution  $\varphi$  found so far
3: improvement  $\leftarrow true$ 
4: while improvement do
5:   while improvement do
6:     improvement  $\leftarrow false$ 
7:     for  $i = \{1, \dots, n\}$  do
8:       randomly select  $j$  from  $\{1, \dots, n\}$  with  $j \neq i$ 
9:        $\varphi' \leftarrow \varphi \oplus \text{swap}(i, j)$ 
10:      if  $\text{IsImprovementMove}(\varphi, \varphi')$  then
11:         $\varphi \leftarrow \varphi'$ 
12:        improvement  $\leftarrow true$ 
13:      end if
14:    end for
15:  end while
16:  for  $i = \{1, \dots, n\}$  do
17:    randomly select  $j$  from  $\{1, \dots, n\}$  with  $j \neq i$ 
18:     $\varphi' \leftarrow \text{insert}(i, j) \oplus \varphi$ 
19:    if  $\text{IsImprovementMove}(\varphi, \varphi')$  then
20:       $\varphi \leftarrow \varphi'$ 
21:      improvement  $\leftarrow true$ 
22:    end if
23:  end for
24:  if improvement  $= false$  and  $VS(G, \varphi) < VS(G, \varphi^*)$  then
25:    for  $i = \{1, \dots, n\}$  do
26:      for  $j = \{i+1, \dots, n\}$  do
27:         $\varphi' \leftarrow \text{swap}(i, j) \oplus \varphi$ 
28:        if  $\text{IsImprovementMove}(\varphi, \varphi')$  then
29:           $\varphi \leftarrow \varphi'$ 
30:          improvement  $\leftarrow true$ 
31:        end if
32:      end for
33:    end for
34:  end if
35: end while

```

better than φ (i.e., $\text{IsImprovementMove}(\varphi, \varphi')$ returns true), φ is replaced by φ' . This procedure is repeated until there is no further improvement (lines 5 to 15). Note that it does not guarantee that swapping any two vertices in φ can improve the current solution. Subsequently, the algorithm traverses the n -sized insert neighborhood in the similar way as the swap neighborhood (lines 16 to 23). Finally, if both the n -sized swap neighborhood and n -sized insert neighborhood cannot improve the current solution (i.e., the flag **improvement** is *false*), and the objective function value of the current solution is smaller than that of the best found solution (line 24), all the moves of swapping any two different vertices are examined and the best move is performed until it cannot improve the current solution any more (lines 25 to 33).

TABLE 1. Summary of the properties of the benchmark instance sets.

Name	Num.	$ V $	$ E $	opt.
HB	62	24 ~ 960	34 ~ 3721	No
Grids	50	25 ~ 2916	40 ~ 5724	Yes
Trees	50	22 ~ 202	21 ~ 201	Yes

One observes that Algorithm 3 mainly works on the n -sized swap neighborhood. The n -sized insert neighborhood is adopted only when there is no further improvement in the swap neighborhood. The n^2 -sized swap neighborhood is applied when there is no further improvement in both the n -sized swap and the n -sized insert neighborhoods, and the current solution is better than the best found solution. This search strategy has the advantage of improving the computational efficiency since searching the n^2 -sized swap neighborhood is time consuming.

G. QUALITY AND DISTANCE GUIDED LOCAL SEARCH IN QD-ILS

In order to drive the search into unvisited promising regions, we adopt the quality and distance guided local search method (QD-LS) as the diversification strategy (see Section II-A). For the VSP problem, the distance between two solutions are defined as the average sum of the difference of the positions of all the vertices in the two solutions. Specifically, given the current solution $\varphi = (v_1, \dots, v_n)$ and the best found solution $\varphi^* = (v_1^*, \dots, v_n^*)$, $\varphi(v_i)$ is the position of vertex v_i in φ , i.e., $\varphi(v_i) = i$, and $|\varphi(v_i) - \varphi^*(v_i)|$ is the difference of the positions of vertex v_i in the two solution φ and φ^* . Therefore, the distance between two solutions φ and φ^* , which is denoted by $d(\varphi, \varphi^*)$, can be formally defined as:

$$d(\varphi, \varphi^*) = \frac{1}{n} \sum_{i=1}^n |\varphi(v_i) - \varphi^*(v_i)| \quad (2)$$

The difference of the quality and distance guided local search and the basic local search is the objective function. By adding the distance between the current solution and the best found solution to the original objective function, the quality and distance guided local search strives the search to unexplored areas of the search space.

Algorithm 4 gives the pseudo-code of the quality and distance guided local search. Compared with the basic local search procedure, this procedure works on the n -sized swap and insert neighborhood. Besides, the criterion of judging whether a new solution is better than the current solution is the function F , which considers both the solution quality and the distance between the current solution and the best found solution.

III. COMPUTATIONAL RESULTS

In this section, extensive experimental results are reported by applying QD-ILS to tackle the standard benchmark instances of VSP available in the literature and the performance of

Algorithm 4 Pseudo-Code of the Quality and Distance Guided Local Search Procedure (QD_LS)

```

1: Input: The graph  $G$ , the best found solution  $\varphi^*$ , and the
   solution  $\varphi$  to be optimized
2: Output: An optimized solution  $\varphi$  found so far
3: improvement  $\leftarrow true$ 
4: while improvement do
5:   while improvement do
6:     improvement  $\leftarrow false$ 
7:     for  $i = \{1, \dots, n\}$  do
8:       randomly select  $j$  from  $\{1, \dots, n\}$  with  $j \neq i$ 
9:        $\varphi' \leftarrow \varphi \oplus swap(i, j)$ 
10:      if  $F(\varphi') < F(\varphi)$  then
11:         $\varphi \leftarrow \varphi'$ 
12:      improvement  $\leftarrow true$ 
13:    end if
14:  end for
15: end while
16: for  $i = \{1, \dots, n\}$  do
17:   randomly select  $j$  from  $\{1, \dots, n\}$  with  $j \neq i$ 
18:    $\varphi' \leftarrow insert(i, j) \oplus \varphi$ 
19:   if  $F(\varphi') < F(\varphi)$  then
20:      $\varphi \leftarrow \varphi'$ 
21:   improvement  $\leftarrow true$ 
22: end if
23: end for
24: end while

```

QD-ILS is compared with that of the state-of-the-art algorithms for VSP.

A. EXPERIMENTAL PROTOCOL AND BENCHMARK INSTANCES

Our QD-ILS algorithm is programmed in C and runs on an Intel Xeon E5440 processor with 2.83 GHz CPU and 2 GB RAM. In our experiments, we set β to -4.5 (Section III-D presents the experimental results to analyze the impact and determine the value of parameter β). The maximum run-time limit for QD-ILS in our experiments is 1,000 seconds, which is the same as the time settings used in general variable neighborhood search (GVNS) algorithm proposed in [30].

We evaluate the performance of our QD-ILS algorithm on three sets of problem instances, in total 162 instances. All instances are available at <http://www.opticom.es/vsp/>. The detailed descriptions of each type of the benchmark sets are given below:

- **HB:** 62 instances are derived from the Harwell-Boeing Sparse Matrix Collection. This collection consists of a set of standard test matrices $M = M_{uv}$ arising from problems in linear systems, least squares, and engineering disciplines. The graphs are derived from these matrices by considering an edge (u, v) for every element $M_{uv} = 0$. From the original set we have selected the 73 graphs with $n \leq 1000$. The number of vertices and edges range from 24 to 960 and from 34 to 3721, respectively.

TABLE 2. Summarized results over the sets Grids, Trees, and HB.

Alg.	Grids				Trees				HB			
	N_{opt}	Avg.	Dev.(%)	t_{avg}	N_{opt}	Avg.	Dev.(%)	t_{avg}	N_{best}	Avg.	Dev.(%)	t_{avg}
GVNS	50	29.5	0.00	93.32	40	4.30	4.00	9.07	35	24.60	6.42	482.08
ILS	50	29.5	0.00	28.58	31	4.52	10.24	49.23	30	25.79	11.58	384.79
QD-ILS	50	29.5	0.00	8.68	46	4.18	1.95	6.77	58	23.21	0.42	243.13

- **Grids:** This set consists of 50 matrices constructed as the Cartesian product of two paths [35]. They are also called two dimensional meshes and the optimal solution of the VSP for squared grids is known by construction [28]. Specifically, the vertex separation value of a square grid of size $\lambda \times \lambda$ is λ . For this set, the vertices are arranged on a square grid with a dimension $\lambda \times \lambda$ for $5 \leq \lambda \leq 54$. The number of vertices and edges ranges from $5 \times 5 = 25$ to $54 \times 54 = 2916$ and from 40 to 5724, respectively.
- **Trees:** Let $T(\lambda)$ be the set of trees with the minimum number of nodes and vertex separation equal to λ . As stated in [14], there is just one tree in $T(1)$, namely the tree with a single edge, and another one in $T(2)$, the tree constructed with a new node acting as root of three subtrees that belong to $T(1)$. In general, to construct a tree with vertex separation $\lambda + 1$, it is necessary to select any three members from $T(\lambda)$ and link any one node from each of these to a new node acting as the root of the new tree. The number of nodes, $n(\lambda)$, of a tree in $T(\lambda)$ can be obtained via using the recurrence relation $n(\lambda) = 3n(\lambda - 1) + 1$ where $n(1) = 2$. We consider 50 different trees: 15 trees in $T(3)$, 15 trees in $T(4)$ and 20 trees in $T(5)$. The number of vertices and edges ranges from 22 to 202 and from 21 to 201, respectively.

Table 1 summarizes the properties of the benchmark instance sets mentioned above, where column *Num.* represents the number of instances in the set, columns $|V|$ and $|E|$ give the range of number of vertices and edges of the instances in the set, respectively, and column *opt.* indicates whether the optimal objective value are known for the set of instances.

B. COMPUTATIONAL RESULTS ON BENCHMARK INSTANCES

Our QD-ILS algorithm is compared with the current state-of-the-art metaheuristic GVNS [30] on the benchmark instances. Tables 3 and 4 report the detailed computational results obtained by QD-ILS, ILS, and GVNS on HB and Trees sets, respectively. ILS is a modified version of QD-ILS, which is obtained by replacing the QD-LS with a simple perturbation procedure and adopting an acceptance criterion procedure. The perturbation procedure used in ILS is similar to that in [29], which selects $0.04|V|$ vertices from V with the descending order of cut value, and then each selected vertex is exchanged with another randomly selected vertex.

The acceptance criterion used in ILS is quite simple: the best local optimum found so far or the most recently generated local optimum is accepted as the next starting solution when the number of consecutive non-improving iterations exceeds 200. The acceptance criterion and parameter settings are also chosen by extensive preliminary experiments.

As mentioned in [30], the instances in Grids set are easily solved for GVNS as well as QD-ILS, so we do not report the detailed computational results for Grids instances. In Table 3, column f_{best} gives the best found objective value. In Table 4, column f_{opt} gives the optimal objective value. Columns f and t are the best objective value and corresponding computational time (in seconds) obtained by GVNS. Columns f_{min} , f_{avg} , and t_{avg} show the best results, the average results, and the average computational time (in seconds) obtained by each algorithm. The results of GVNS are obtained on an Intel Core i7 2600 3.4 GHz CPU and 4 GB RAM. Since no sufficient information is available to benchmark the computers used in [30], the reported computational times are given mainly for indicative purposes.

Table 2 summarizes the results of both QD-ILS and GVNS on the sets Grid, Trees, and HB instances. For each set, columns N_{opt} and N_{best} show the number of optimal solutions found and the best solution found, respectively. Column *Avg.* shows the average objective value over all instances. Column *Dev.(%)* shows the average deviation with respect to the best or optimal solution. Column t_{avg} shows the average computational time in seconds used by each procedure.

From Tables 2, 3, and 4, one observes that our QD-ILS outperforms GVNS in both solution quality and computational efficiency. Specifically, for the Grids set, although both QD-ILS and GVNS can optimally solve all the instances, QD-ILS requires less computational time than GVNS. For the Tree set, QD-ILS obtains the optimal solutions for six more instances than GVNS, and requires less computational time. For the most challenging 62 instances in HB set, QD-ILS can improve the previous best known results for 27 instances (in bold), match the previous best known results for 31 cases, and fail to obtain the previous best known results for only 4 cases (with underline). Besides, QD-ILS requires less computational time than GVNS.

C. IMPACT OF PARAMETER β

In this section, we present the merits of the quality and distance guided local search in QD-ILS. First, we analyze

TABLE 3. Comparison between QD-ILS and GVNS on the HB benchmark instances within 1,000 vertices.

Instance	f_{best}	GVNS		QD-ILS			ILS		
		f	t	f_{min}	f_{avg}	t_{avg}	f_{min}	f_{avg}	t_{avg}
494 BUS	14	14	1000.48	14	17.32	219.9	14	18	262.97
662 BUS	21	21	1001.26	21	24.25	337.65	22	26.05	533
685 BUS	21	30	1000.44	21	23.8	504.35	22	24.9	668.79
ASH292	15	18	229.23	15	16.65	267.06	16	18.2	370.17
ASH85	8	8	0.78	8	8.2	1.48	8	8.05	1.56
BCSPWR01	4	4	0.03	4	4	0.02	4	4	0.07
BCSPWR02	3	5	0.15	3	3.85	0.45	3	3.85	0.27
BCSPWR03	7	7	3.87	7	8.2	2.14	7	8	12.87
BCSPWR04	13	14	205.59	13	14.85	141.04	14	15.1	343
BCSPWR05	12	12	1000.44	12	14.05	174.65	12	15.95	247.28
BCSSTK01	13	15	0.3	13	13.4	1.38	13	13.55	2.11
BCSSTK02	65	65	2.69	65	65	0.14	65	65	0.03
BCSSTK04	34	34	87.92	34	34	62.31	34	34	81.26
BCSSTK05	20	20	18.75	20	20.1	84.78	20	20.05	124.63
BCSSTK06	41	41	1003.25	41	45.6	570.23	45	51.85	406.57
BCSSTK07	32	41	1001.48	32	43.15	691.45	45	50.9	570.48
BCSSTK19	14	14	1000.27	<u>15</u>	20.65	994.49	46	71.25	859.65
CAN 144	8	8	4.7	8	9.9	21.42	10	10	0.04
CAN 161	20	24	18.6	20	23.6	76.1	24	24.3	74.93
CAN 187	14	14	37.83	14	14.1	28.34	14	14.8	88.93
CAN 229	21	22	234.34	21	22.5	156.72	22	23.1	245.51
CAN 256	24	26	987.43	24	27.5	302.1	25	28.95	446.76
CAN 268	24	25	586.68	24	25.6	367.1	25	26.95	416.85
CAN 292	24	26	1000.89	24	25.3	280.22	26	28.55	340.29
CAN 445	49	53	1000.78	49	51.75	444.61	59	63.5	536.8
CAN 634	65	73	1000.83	65	76.85	858.9	76	82.35	804.97
CAN 715	59	59	1008.7	59	65.75	640.29	59	69.6	863.89
CAN 24	5	5	0.01	5	5	0.05	5	5	0.02
CAN 61	8	8	0.58	8	8.2	0.43	8	8.3	10.01
CAN 62	4	5	0.16	4	4.15	2.96	5	5.15	0.83
CAN 73	10	12	0.82	10	10	0.27	10	10	0.47
CAN 96	14	16	3	14	14.45	12.76	16	18.55	4.46
DWT 162	10	13	8.78	10	11.3	51.14	15	15.7	23.75
DWT 193	29	37	189.33	29	30.85	223.19	39	42.4	349.83
DWT 209	18	18	109.71	18	18.25	84.71	18	20.9	98.72
DWT 221	11	11	102.43	11	11.95	151.75	11	12.65	267.29
DWT 245	12	16	225.48	12	14.4	168.74	13	14.75	237.44
DWT 307	26	30	578.97	26	27.8	286.66	34	38.45	267.99
DWT 310	11	11	61.48	11	14.55	51.99	11	14.95	305.87
DWT 361	15	19	128.54	15	17.5	92.11	19	23.9	54.65
DWT 419	21	21	1000.16	22	26.45	526.57	24	27.3	543.54
DWT 503	47	49	1000.76	47	53.45	434.7	54	60.75	747.25
DWT 592	27	27	1000.13	27	38.5	926.08	31	47.85	1142.58
DWT 758	20	21	1000.87	20	20	5.51	20	20	4.68
DWT 869	32	34	1000.72	32	40.15	32.54	34	42.55	814.88
DWT 878	30	35	1002.76	30	35.1	360.94	36	37.5	324.75
DWT 918	34	34	1002.8	<u>35</u>	42.35	638.89	40	54.35	875.94
DWT 59	5	5	0.21	5	5.4	0.2	5	5.75	3.7
DWT 66	2	2	0.17	2	2.95	1	3	3	0.01
DWT 72	3	3	0.2	3	3.95	0.44	3	3.95	0.54
DWT 87	8	8	0.92	8	8.1	0.65	8	8.1	3.28
GR 30 30	42	45	1025.42	42	47	943.51	45	49.95	877.96
LSHP 265	17	17	57.14	17	17	23.33	17	17	20.01
LSHP 406	21	21	339.5	21	21.05	155.47	21	21	196.71
LSHP 577	25	25	1001	25	25.25	326.75	25	25	269.32
LSHP 778	29	29	1005.24	29	29.15	616.25	29	29.15	532.93
NOS3	73	76	1016.35	73	85.75	35.58	76	94.5	26.22
NOS4	8	8	1.05	8	8.1	3.15	8	8.1	1.87
NOS5	56	56	1001.86	56	59.5	557.02	56	61.25	296.73
NOS6	15	15	789.73	<u>16</u>	23.45	15.26	30	30	1.13
NOS7	65	65	794.71	65	66.35	561.04	65	66.05	548.52
PLAT362	35	35	1000.17	35	36.3	553.15	35	42.8	499.58

TABLE 4. Comparison between QD-ILS and GVNS on the Trees benchmark instances within 1,000 vertices.

Instance	path	f_{opt}	GVNS		QD-ILS			ILS		
			f	t	f_{min}	f_{avg}	t_{avg}	f_{min}	f_{avg}	t_{avg}
TREE_202_5_rot1	1rot	5	6	19.5	5	6.35	18.36	5	6.95	133.3
TREE_202_5_rot2	1rot	5	6	19.44	5	6.55	16.18	6	7.25	133.33
TREE_202_5_rot3	1rot	5	6	23.72	5	6.8	22.34	6	7	63.21
TREE_202_5_rot4	1rot	5	6	19.46	6	7.25	3.33	6	7	108.51
TREE_202_5_rot56	1rot	5	6	10.6	5	6.7	48.6	6	8.05	244.75
TREE_202_5_rot57	1rot	5	6	10.48	6	7.05	19.49	6	7.25	380.87
TREE_202_5_rot58	1rot	5	6	10.56	6	7.75	4.06	6	7.9	12.67
TREE_202_5_rot59	1rot	5	6	10.82	5	6.9	17.89	8	8	0.02
TREE_202_5_rot60	1rot	5	6	10.75	5	7.15	13.24	6	7.15	181.71
TREE_22_3_rot1	1rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot2	1rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot3	1rot	3	3	0.01	3	3	0.01	3	3	0.01
TREE_22_3_rot4	1rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot5	1rot	3	3	0.01	3	3	0.01	3	3	0.01
TREE_67_4_rot1	1rot	4	4	0.78	4	4	0.73	4	4	2.65
TREE_67_4_rot2	1rot	4	4	0.78	4	4	1.57	4	4	1.18
TREE_67_4_rot3	1rot	4	4	0.37	4	4	0.18	4	4	0.73
TREE_67_4_rot4	1rot	4	4	0.77	4	4	0.61	4	4	1.71
TREE_67_4_rot5	1rot	4	4	0.37	4	4	0.22	4	4	3.47
TREE_202_5_rot10	2rot	5	5	23.71	5	6.35	15.35	6	6.8	58.42
TREE_202_5_rot56	2rot	5	5	11.41	5	7.15	18.07	6	7.55	288.77
TREE_202_5_rot57	2rot	5	6	31.3	6	7.5	24.99	6	7.85	384.18
TREE_202_5_rot59	2rot	5	5	25.34	5	6.45	14.66	6	7.7	84.3
TREE_202_5_rot6	2rot	5	5	28.75	5	6.2	8.88	6	7.45	107.63
TREE_202_5_rot60	2rot	5	5	36.32	5	6.05	19.9	6	6.75	61.81
TREE_202_5_rot7	2rot	5	5	47.38	5	6.7	21.6	6	7.25	14.58
TREE_202_5_rot8	2rot	5	5	23.73	5	7.25	8.17	6	7.45	10.24
TREE_202_5_rot9	2rot	5	5	47.57	5	6.45	11.37	6	6.9	74.7
TREE_22_3_rot1	2rot	3	3	0.01	3	3	0.01	3	3	0.01
TREE_22_3_rot2	2rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot3	2rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot4	2rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot5	2rot	3	3	0.01	3	3	0.01	3	3	0
TREE_67_4_rot10	2rot	4	4	0.73	4	4.05	0.91	4	4	3.32
TREE_67_4_rot6	2rot	4	4	0.38	4	4	0.43	4	4	3.63
TREE_67_4_rot7	2rot	4	4	0.6	4	4	1.27	4	4	2.49
TREE_67_4_rot8	2rot	4	4	0.72	4	4	1.73	4	4.05	1.86
TREE_67_4_rot9	2rot	4	4	0.6	4	4.05	1.24	4	4	5.63
TREE_202_5_rot11	3rot	5	5	25.29	5	5.45	0.56	6	6	0.03
TREE_202_5_rot57	3rot	5	5	9.57	5	6.1	0.96	6	6	0.02
TREE_22_3_rot1	3rot	3	3	0.01	3	3	0.01	3	3	0.01
TREE_22_3_rot2	3rot	3	3	0.01	3	3	0.01	3	3	0.01
TREE_22_3_rot3	3rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot4	3rot	3	3	0.01	3	3	0.01	3	3	0
TREE_22_3_rot5	3rot	3	3	0.01	3	3	0.01	3	3	0
TREE_67_4_rot11	3rot	4	4	0.31	4	4.4	4.12	4	4.65	34.58
TREE_67_4_rot12	3rot	4	4	0.32	4	4.5	3.85	4	4.4	37.72
TREE_67_4_rot13	3rot	4	4	0.31	4	4.6	2.91	4	4.95	5.07
TREE_67_4_rot14	3rot	4	4	0.33	4	4	5.56	4	4	5.93
TREE_67_4_rot15	3rot	4	4	0.31	4	4.05	5.26	4	4	8.6

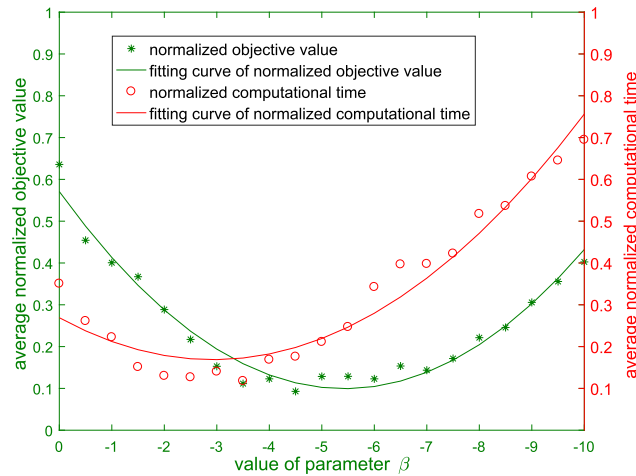


FIGURE 1. The average normalized objective function value and computational time corresponding to different values of parameter β .

the effect of the parameter β on the performance of QD-ILS. For parameter β , we take 21 different values $\beta \in [-10, 0]$ (stepped by 0.5) and keep other parameters fixed and conduct experiments on all the instances in HB, Trees, and Grids sets. We perform 20 independent runs for each parameter and each instance and stop our algorithm when it reaches 10000 generations. Fig. 1 shows the average normalized objective value and computational time¹ and their quadratic fitting curve corresponding to different values of parameter β .

From Fig. 1, one observes that QD-ILS can find relatively high quality solutions when $\beta \in [-7, -3]$, while the corresponding average computational time slightly decreases when β decreases from 0 to -4 , and rapidly increases when $\beta < -4$. This illustrates that larger proportion of distance between the current solution and the best found solution (i.e., with smaller value of β) in the augmented objective function can introduce a larger diversification into the search but requires more computational time. Considering both solution quality and computational time, β is suggested to be -4.5 .

D. COMPARISON BETWEEN QD-ILS AND ILS

In order to further explore the behaviour of QD-ILS, we perform 20 independent runs of QD-ILS and ILS on instance NOS5 and plot the best solutions found by the two algorithms corresponding to the average required computational time in Fig. 2. One finds that the curve of QD-ILS decreases almost the same as that of ILS at early stage of the search, but it outperforms ILS in the long run for it can continue to decrease dramatically and smoothly. Specifically, ILS is less likely to jump out of local optima to obtain better solution for its curve

¹ Average normalized objective value: First, find the maximum and minimum objective value (denoted by f_{max} and f_{min}) over the 20 runs for each instance. Second, normalized the objective value of each run for each instance using $norm.f = (f - f_{min}) / (f_{max} - f_{min})$. Third, average the $norm.f$ of the test instances for each value of parameter β . The average normalized computational times are calculated in the same way.

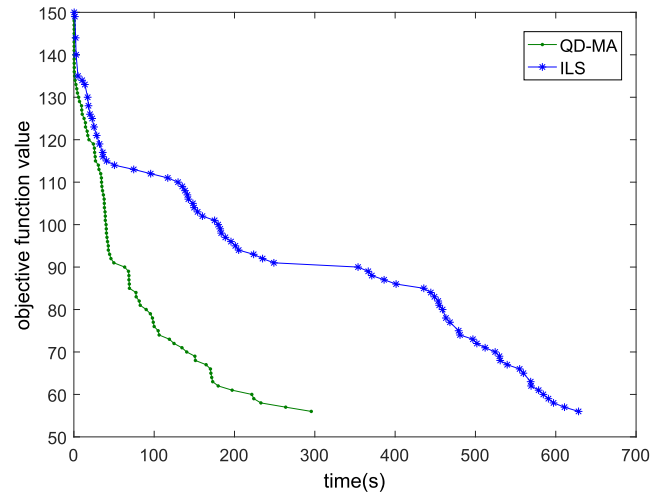


FIGURE 2. Comparison between QD-ILS and ILS on instance nos5 with respect to the best found solution and computational time.

TABLE 5. Wilcoxon's test results of QD-ILS versus ILS in different time scales.

time	R-	R+	z-value	p-value	Significant
1 min	756.5	372.5	-1.5732	0.16	No
5 min	134	1141	-4.8604	0.00	Yes
10 min	50	1175	-5.5954	0.00	Yes
15 min	42.5	1232.5	-5.7235	0.00	Yes

usually keeps flat in a long period. This demonstrates the effectiveness of QD-LS as a diversification mechanism.

Furthermore, to detect the differences between QD-ILS and ILS, we conduct a Wilcoxon's test [36] in four different time scales: 1, 5, 10, and 15 minutes. The statistical test results are reported in Table 5, where R+ is the sum of ranks for the functions on which QD-ILS outperforms ILS, and R- is the sum of ranks for the opposite. Computational results show that the number of the samples with non-zero difference is larger than 10, the sampling distribution of $W = \min \{R-, R+\}$ is a reasonably close approximation of the normal distribution, so we use the z-value to evaluate the hypothesis [37]. If the resulting $|z|$ -value is greater than the critical z-value at a specific level of significance, the Wilcoxon's test detects significant differences between the two algorithms. Since the critical value of z at the significance of 0.05 is 1.96 (i.e., $z_{0.05} = 1.96$), we observe that the $|z|$ value is larger than 1.96 when the run time is larger than 5 minutes, which means that QD-ILS outperforms ILS in the long term of run time.

IV. CONCLUSION

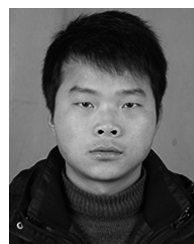
This paper presents QD-ILS, a quality and distance guided iterated local search algorithm for solving the vertex separation problem, which demonstrate the efficacy of the diversification strategy: the quality and distance guided local search.

QD-LS uses the augmented evaluation function which considers both solution quality and distance between the current solution and the best found solution to guide the search to explore promising regions of the search space. QD-ILS combines a basic local search procedure which intensifies the search with a quality and distance guided local search which encourages the search process to examine unvisited regions and to generate solutions that differ from the previously visited solutions with a suitable distance.

Tested on the 162 benchmark instances with up to 2916 vertices, QD-ILS can improve the previous best known results for 33 instances, obtain worse results for 4 instances and match the previous best known results for the others compared with the state-of-the-art algorithms in the literature, demonstrating the efficacy of our proposed algorithm in terms of both solution quality and computational efficiency. The success of QD-ILS on VSP is another support of the effectiveness of the quality and distance guided local search as a diversification mechanism. We expect to apply this powerful diversification strategy to other challenging combinatorial optimization problems in the near future.

REFERENCES

- [1] C. E. Leiserson, "Area-efficient graph layouts," in *Proc. 21st Annu. Symp. Found. Comput. Sci.*, Oct. 1980, pp. 270–281.
- [2] D. D. Kouvatsos, "Mathematical methods for the modular design of computer systems," *Comput. Oper. Res.*, vol. 4, no. 1, pp. 55–63, 1977.
- [3] H. Bodlaender, J. Gustedt, and J. A. Telle, "Linear-time register allocation for a fixed number of registers," in *Proc. SODA*, 1998, pp. 574–583.
- [4] V. Dujmovi et al., "On the parameterized complexity of layered graph drawing," in *Algorithmic*, vol. 52. New York, NY, USA: Springer, 2008, pp. 267–292.
- [5] I. H. Osman, "A tabu search procedure based on a random roulette diversification for the weighted maximal planar graph problem," *Comput. Oper. Res.*, vol. 33, no. 9, pp. 2526–2546, 2006.
- [6] C.-C. Lin, Y.-Y. Lee, and H.-C. Yen, "Mental map preserving graph drawing using simulated annealing," *Inf. Sci.*, vol. 181, no. 19, pp. 4253–4272, 2011.
- [7] N. G. Kinnarsley, "The vertex separation number of a graph equals its pathwidth," *Inf. Process. Lett.*, vol. 42, no. 6, pp. 345–350, 1992.
- [8] L. M. Kirousis and C. H. Papadimitriou, "Searching and pebbling," *Theor. Comput. Sci.*, vol. 47, pp. 205–218, Jan. 1986.
- [9] L. M. Kirousis and C. H. Papadimitriou, "Interval graphs and searching," *Discrete Math.*, vol. 55, no. 2, pp. 181–184, 1985.
- [10] A. Linhares and H. H. Yanasse, "Connections between cutting-pattern sequencing, VLSI design, and flexible machines," *Comput. Oper. Res.*, vol. 29, no. 12, pp. 1759–1772, 2002.
- [11] E. Mäkinen, T. Poranen, and P. Vuorenmaa, "A genetic algorithm for determining the thickness of a graph," *Inf. Sci.*, vol. 138, no. 1, pp. 155–164, 2001.
- [12] M. C. V. Nascimento, F. M. B. Toledo, and A. C. P. L. F. de Carvalho, "Investigation of a new GRASP-based clustering algorithm applied to biological data," *Comput. Oper. Res.*, vol. 37, no. 8, pp. 1381–1388, 2010.
- [13] M. R. Fellows and M. A. Langston, "On search decision and the efficiency of polynomial-time algorithms," in *Proc. 21st Annu. ACM Symp. Theory Comput.*, 1989, pp. 501–512.
- [14] J. A. Ellis, I. H. Sudborough, and J. S. Turner, "The vertex separation and search number of a graph," *J. Inf. Comput.*, vol. 113, no. 1, pp. 50–79, 1994.
- [15] K. Skodinis, "Computing optimal strategies for trees in linear time," in *Proc. 8th Annu. Eur. Symp. Algorithms*, 2000, pp. 403–414.
- [16] S.-L. Peng, C.-W. Ho, T.-S. Hsu, M.-T. Ko, and C. Y. Tang, "A linear-time algorithm for constructing an optimal node-search strategy of a tree," in *Computing and Combinatorics*. Berlin, Germany: Springer, 1998, pp. 279–289.
- [17] J. Ellis and M. Markov, "Computing the vertex separation of unicyclic graphs," *J. Inf. Comput.*, vol. 192, no. 2, pp. 123–161, 2004.
- [18] H. L. Bodlaender and R. H. Möhring, "The pathwidth and treewidth of cographs," *SIAM J. Discrete Math.*, vol. 6, no. 2, pp. 181–188, 1993.
- [19] B. Bollobás and I. Leader, "Edge-isoperimetric inequalities in the grid," *Combinatorica*, vol. 11, no. 4, pp. 299–314, 1991.
- [20] H. L. Bodlaender, T. Kloks, and D. Kratsch, "Treewidth and pathwidth of permutation graphs," *SIAM J. Discrete Math.*, vol. 8, no. 4, pp. 606–616, 1995.
- [21] H. L. Bodlaender, F. V. Fomin, A. M. C. A. Koster, D. Kratsch, and D. M. Thilikos, "On exact algorithms for treewidth," in *Proc. Eur. Symp. Algorithms*, 2006, pp. 672–683.
- [22] H. L. Bodlaender, F. V. Fomin, A. M. C. A. Koster, D. Kratsch, and D. M. Thilikos, "A note on exact algorithms for vertex ordering problems on graphs," *Theory Comput. Syst.*, vol. 50, no. 3, pp. 420–432, 2012.
- [23] P. Crescenzi, R. Grossi, M. Habib, L. LANZI, and A. Marino, "On computing the diameter of real-world undirected graphs," *Theor. Comput. Sci.*, vol. 514, pp. 84–95, Nov. 2013.
- [24] Y. Kobayashi, K. Komuro, and H. Tamaki, "Search space reduction through commitments in pathwidth computation: An experimental study," in *Proc. Int. Symp. Experim. Algorithms*, 2014, pp. 388–399.
- [25] D. Coudert, D. Mazauric, and N. Nisse, "Experimental evaluation of a branch-and-bound algorithm for computing pathwidth and directed pathwidth," *J. Experim. Algorithmics*, vol. 21, no. 1, pp. 1–3, 2016.
- [26] Y. Li, Y. Shang, and Y. Yang, "Clustering coefficients of large networks," *Inf. Sci.*, vol. 382, pp. 350–358, Mar. 2017.
- [27] H. L. Bodlaender, J. R. Gilbert, H. Hafsteinsson, and T. Kloks, "Approximating treewidth, pathwidth, frontsize, and shortest elimination tree," *J. Algorithms*, vol. 18, no. 2, pp. 238–255, 1995.
- [28] J. Díaz, J. Petit, and M. Serna, "A survey of graph layout problems," *ACM Comput. Surv.*, vol. 34, no. 3, pp. 313–356, 2002.
- [29] A. Duarte, L. F. Escudero, R. Martí, N. Mladenovic, J. J. Pantrigo, and J. Sánchez-Oro, "Variable neighborhood search for the vertex separation problem," *Comput. Oper. Res.*, vol. 39, no. 12, pp. 3247–3255, 2012.
- [30] J. Sánchez-Oro, J. J. Pantrigo, and A. Duarte, "Combining intensification and diversification strategies in VNS. An application to the vertex separation problem," *Comput. Oper. Res.*, vol. 52, pp. 209–219, Dec. 2014.
- [31] J. Ding, Z. Lü, L. Xu, and T. Zhou, "A quality and distance guided hybrid algorithm for the vertex separator problem," *Comput. Oper. Res.*, vol. 78, pp. 255–266, Feb. 2017.
- [32] F. Glover, "Adaptive memory projection methods for integer programming," in *Metaheuristic Optimization via Memory and Evolution*. New York, NY, USA: Springer, 2005, pp. 425–440.
- [33] F. Glover and S. Hanafi, "Metaheuristic search with inequalities and target objectives for mixed binary optimization—Part II: Exploiting reaction and resistance," in *Modeling, Analysis, and Applications in Metaheuristic Computing: Advancements and Trends*. New York, NY, USA: Springer, 2010.
- [34] E. Rodriguez-Tello, J.-K. Hao, and J. Torres-Jimenez, "An effective two-stage simulated annealing algorithm for the minimum linear arrangement problem," *Comput. Oper. Res.*, vol. 35, no. 10, pp. 3331–3346, 2008.
- [35] A. Raspaud, H. Schröder, O. Sýkora, L. Torok, and I. Vrt'o, "Antibandwidth and cyclic antibandwidth of meshes and hypercubes," *Discrete Math.*, vol. 309, no. 11, pp. 3541–3552, 2009.
- [36] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.
- [37] F. Mosteller and R. E. K. Rourke, *Sturdy Statistics: Nonparametrics and Order Statistics*. Reading, MA, USA: Addison-Wesley, 1973.



JUNWEN DING received the B.Sc. degree in computer science from Wuhan Textile University, Wuhan, China, in 2011, and the M.Sc. degree in computer software and theory from the Huazhong University of Science and Technology, China, in 2014, respectively. He is currently pursuing the Ph.D. degree with the Laboratory of Smart Computing and Optimization, Huazhong University of Science and Technology.

His current research interests include evolutionary computation, machine scheduling, and airport gate assignment.



tical applications, traveling salesman problem, and vehicle routing problem.

TAOQING ZHOU was born in 1972. He received the master's degree in computer application technology from South-Central University for Nationalities, Wuhan, in 2003. He is currently pursuing the Ph.D. degree with the Huazhong University of Science and Technology. He is currently a Senior Lecturer with Zhejiang A&F University. His research interests include computational intelligence, solving large-scale combinatorial search problems, designing heuristic algorithms for practical applications, traveling salesman problem, and vehicle routing problem.



YUJIN YUAN was born in 1994. He received the B.Sc. degree in computer science from the Huazhong University of Science and Technology, China, in 2016. He is currently pursuing the master's degree with the Digital Media Computing and Design Laboratory, Zhejiang University, China. His current research interests include artificial intelligence and natural language processing.

...



and Technology, and the Director of the Laboratory of Smart Computing and Optimization. His research is in the area of artificial intelligence, computational intelligence, operations research and adaptive metaheuristics for solving large-scale real-world and theoretical combinatorial optimization, and constrained satisfaction problems.

ZHIPENG LÜ received the B.S. degree in applied mathematics from Jilin University, China, in 2001, and the Ph.D. degree in computer software and theory from the Huazhong University of Science and Technology, China, in 2007. He was a Post-Doctoral Research Fellow with LERIA, Department of Computer Science, University of Angers, France, from 2007 to 2011. He is a Professor with the School of Computer Science and Technology, Huazhong University of Science